

LLM 품질 개선, 어디서부터 시작할까?

평가지표(Rubric)와 료프 엔지니어링, Agent 평가

BE 밍트

목차

01

평가 기준 만들기

- 프롬프트만 수정하면 생기는 문제
- 실패 패턴을 Rubric으로 정리

02

평가 자동화하기

- Loop Engineering
- LLM-as-Judge

03

Agent 평가하기

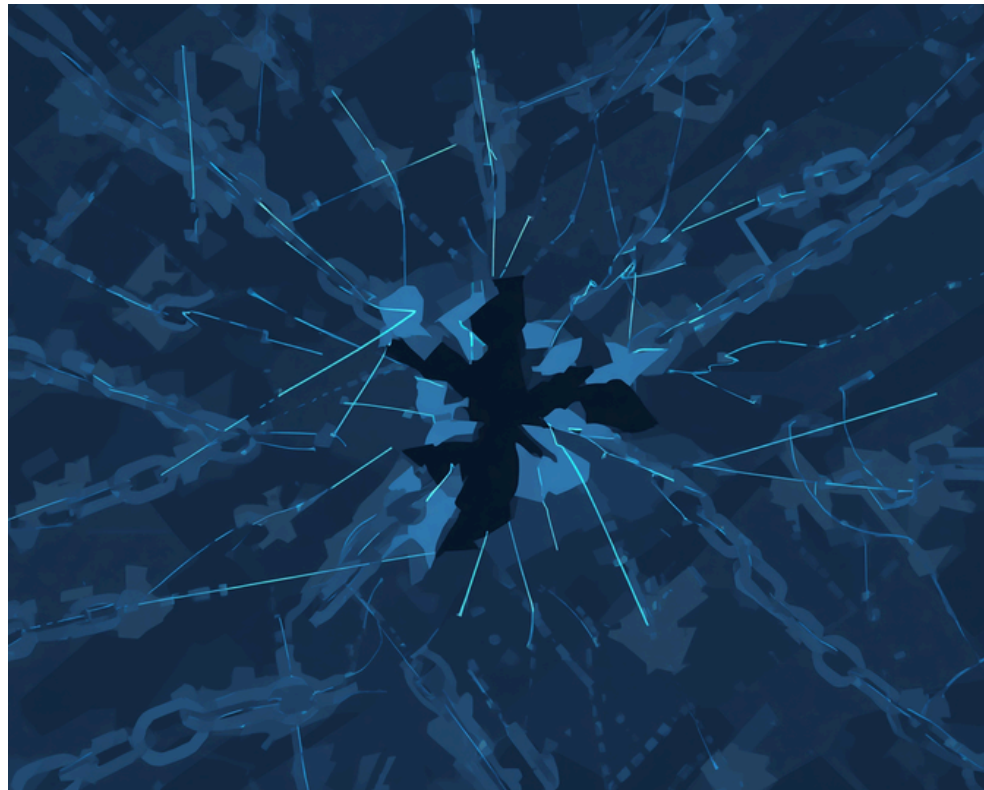
- LLM 서비스와 Agent의 차이
- 과정 평가와 운영하면서 만난 함정

| 1. 평가 기준 만들기

프롬프트를 수정하며 생긴 문제들



😞 수정하면 다른 곳이 나빠진다



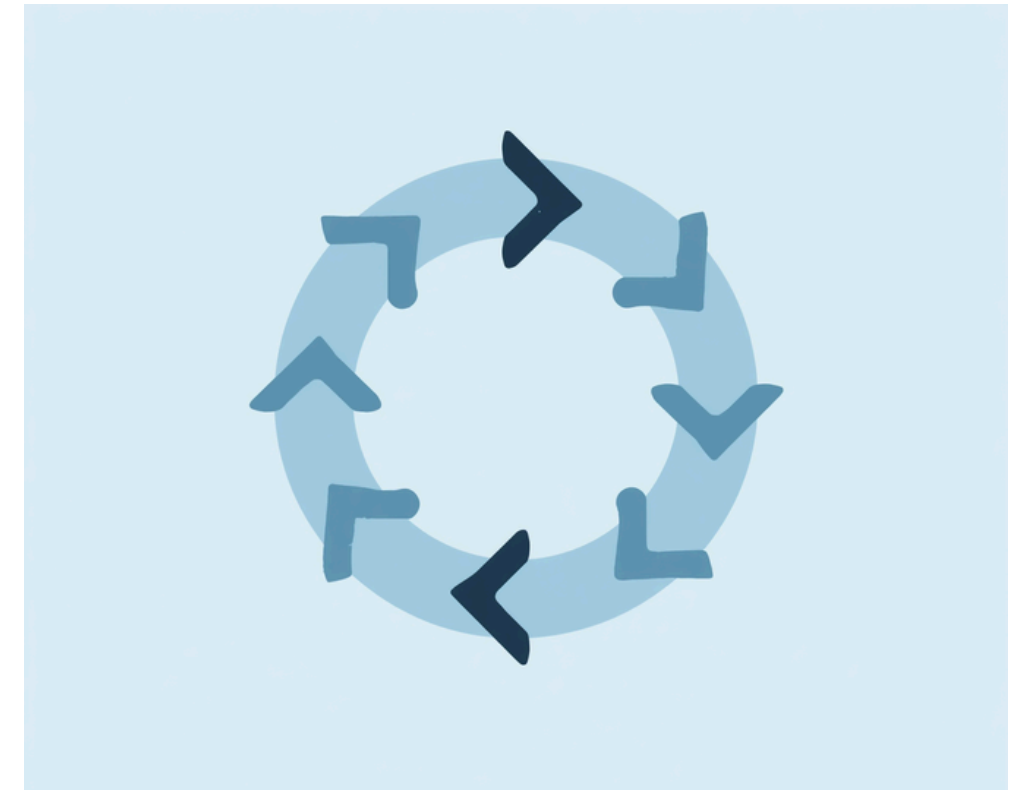
A를 고치면 B에서 문제가 생긴다
→ 수정이 또 다른 수정을 만든다

🔍 좋아졌는지 확인이 안 된다



이전 결과와 감으로 비교한다
→ 개선됐는지 객관적으로 알 수 없다

🔄 같은 문제가 반복된다



고쳤다고 생각했는데 문제가 반복된다
→ 근본 원인이 해결되지 않는다

좋은 결과의 기준부터 정의하자

Failure Patterns → Rubric

실패 패턴을 정리하여 평가 기준(Rubric)을 만든다.

반복된 실패 유형

- 1 입력에 없는 원인을 추론함
- 2 작은 변화를 큰 추세처럼 해석함
- 3 추측인데 사실처럼 씀
- 4 같은 상황인데 표현이 그때그때 달라짐
- 5 문장이 너무 길어서 뭘 말하는지 모르겠음



Rubric

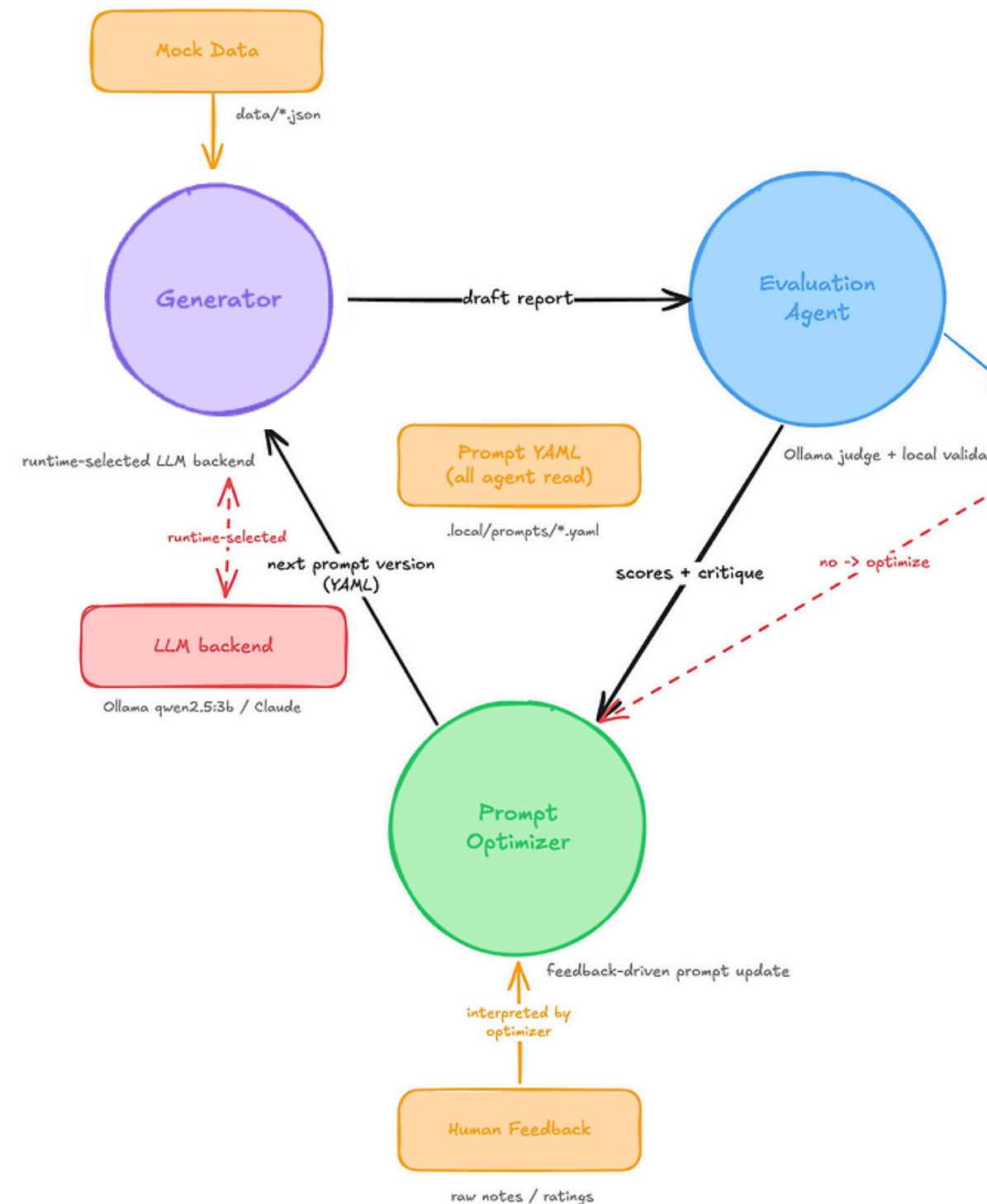
항목	확인한 것
근거성	입력 데이터에 있는 내용만 썼는가
해석 적정성	데이터 이상으로 해석하지 않았는가
불확실성	사실인지 추측인지 구분했는가
표현 일관성	비슷한 상황에서 같은 기준으로 썼는가
가독성	읽었을 때 바로 이해되는가

| 2. 평가 자동화하기

LLM-as-Judge / Loop Engineering로 개선하자

LLM이 다른 LLM의 결과를 채점하는 것

AI가 스스로 수행·검증·개선을 반복하는 구조



Generator

초안 생성

Evaluation Agent

Rubric 기준으로 채점하고 실패 원인 반환

Prompt Optimizer

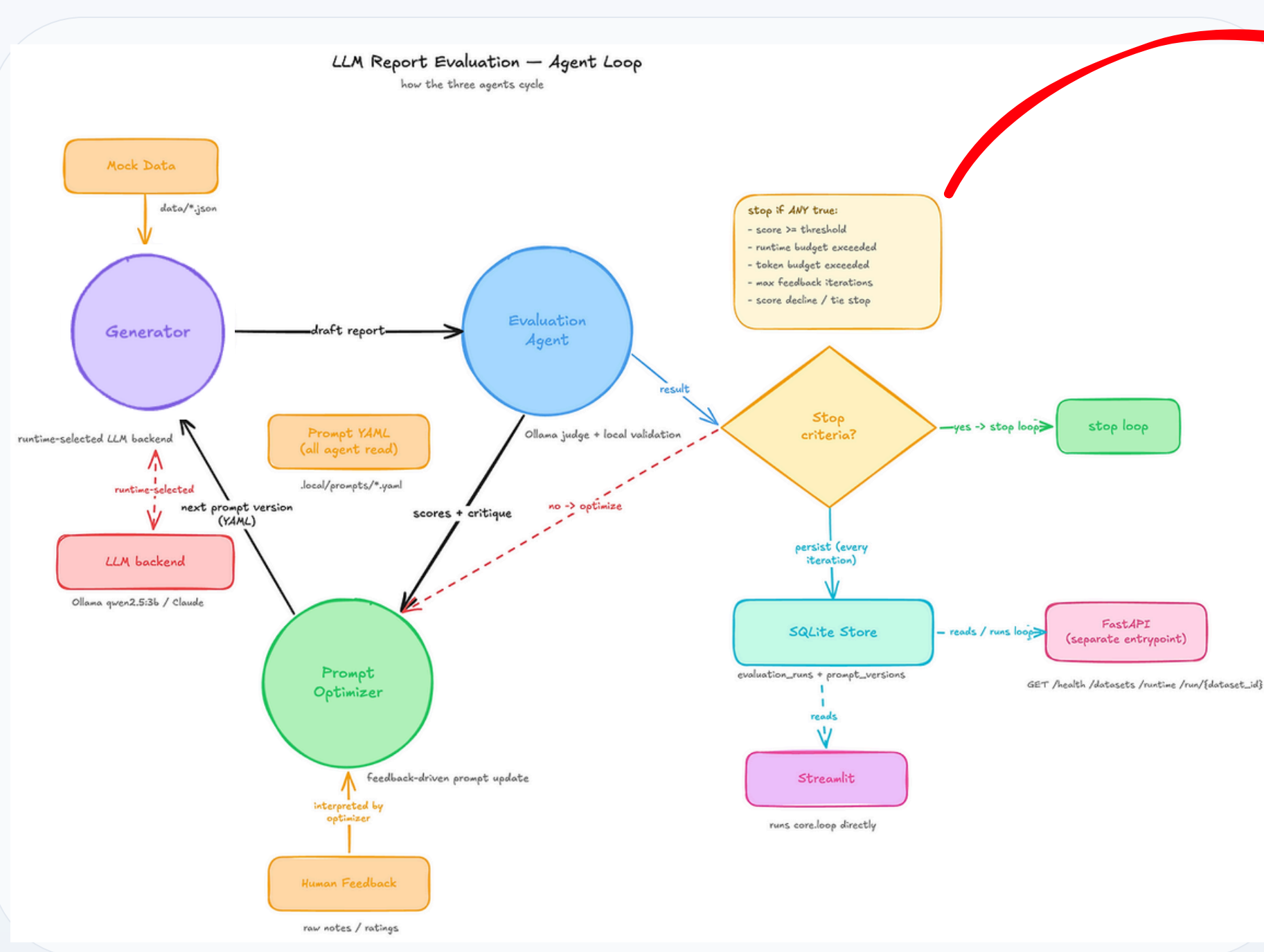
평가 결과와 Human Feedback을 다음 프롬프트로 반영

SQLite State Store

프롬프트와 평가 이력 저장

Loop Engineering

AI가 스스로 수행·검증·개선을 반복하는 구조



루프를 멈추는 경우

- 목표 점수를 만족한 경우
- 이전과 점수가 같거나 오히려 낮아진 경우
- 최대 반복 횟수에 도달한 경우
- 최대 실행 시간을 초과한 경우
- 최대 토큰 사용량을 초과한 경우

Runs

prompt_version	overall_score	groundedness	appropriateness	calibration	consistency	readability
generator_v1@ca684512	4.3625	3.3	4.7	4.5	4.8	4.9
generator_v2@68d41598	4.7375	4.8	4.7	4.5	4.8	4.9
generator_v3@578ed05f	4.875	5	5	4	5	5
generator_v4@6bc95e56	4.875	5	5	4	5	5

Loop Engineering

AI가 스스로 수행·검증·개선을 반복하는 구조

Elapsed seconds

53.9

Runs

prompt_version	overall_score	groundedness	appropriateness	calibration	consistency	readability
generator_v1@ca684512	3.25	1.5	4	3	3	5
generator_v2@c97b4a01	4.125	2	5	4	5	5
generator_v3@62b5dfa5	4.075	1.9	4.8	4.7	4.9	4.8

점수는 목표가 아니라 가드레일이다.

| 3. Agent 평가하기

LLM vs Agent

구분	LLM	Agent
개념	입력을 받아 응답을 생성한다	목표를 달성하기 위해 스스로 판단하고 Tool을 실행하며 반복한다
평가 대상	• 응답이 정확한가 • 비용과 응답 시간이 적절한가	• 올바른 결론을 도출했는가 • 근거를 바탕으로 명확하게 설명했는가 • 필요한 정보를 충분히 수집했는가 • 불필요한 Tool을 호출하지 않았는가 결과 과정
평가 방법	생성된 응답을 정답과 비교한다	결과와 수행 과정을 함께 평가한다
핵심 차이	단일 출력 품질 중심	결과와 목표 달성 과정까지 포함한 종합 평가

Agent는 결과만이 아니라, 목표 달성의 과정까지 평가해야 한다

Result Accuracy

올바른 결론을 도출했는가

LLM-as-Judge

근거를 바탕으로 명확하게 설명했는가

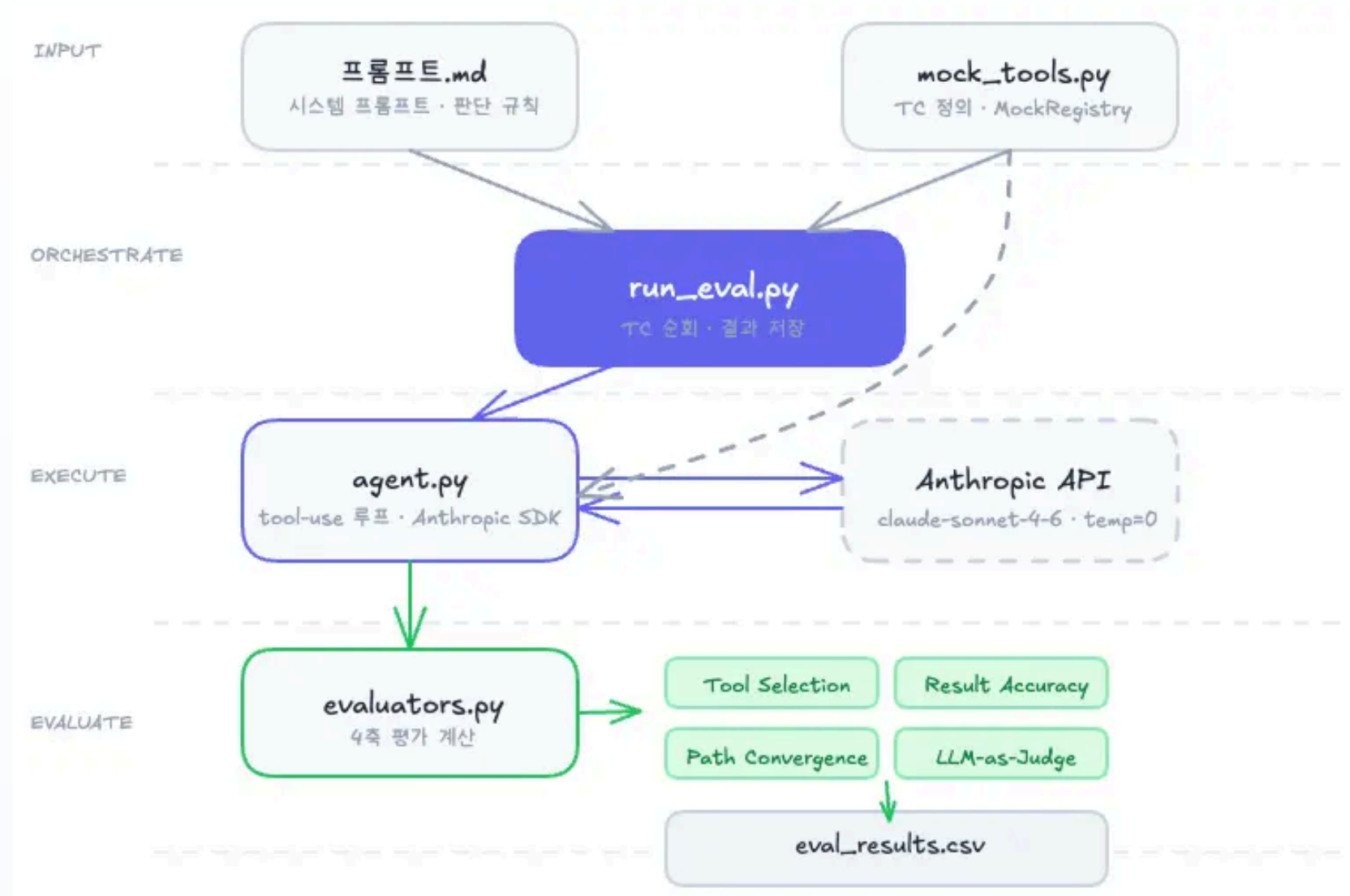
Tool Selection

필요한 정보를 충분히 수집했는가

Path Convergence

불필요한 Tool을 호출하지 않았는가

Agent 평가



mock_tools.py

테스트 케이스와 도구 정의

run_eval.py

테스트 케이스 실행, 결과 저장 담당

agent.py

Agent가 Tool을 호출하며 목표를 달성하는 과정 실행

evaluators.py

네 가지 평가 지표 계산

Agent 평가

과정 지표 결과 지표

TC	Tool Selection	Result Accuracy	Path	Quality
TC-001	1.00	0.67 (2/3)	0.74	9/10
TC-002	1.00	1.00 (1/1)	1.00	10/10
TC-003	1.00	1.00 (3/3)	0.87	9/10
TC-004	1.00	0.33 (1/3)	0.77	9/10
평균	1.00	0.75	0.85	

필요한 도구 모두 호출

분석이 틀린 경우 존재

결과 지표만 봤다면?



정보 수집이 부족한건가?
분석이 틀린건가?



Tool을 추가
(오진)

VS

결과 지표 + 과정 지표



Tool Selection : 정보는 충분
Accuracy : 분석이 틀렸어



판단 규칙을 개선
(정확한 수정)

Agent는 결과만이 아니라, 목표 달성의 과정까지 평가해야 한다

같은 정답이라도 더 빠르게, 더 적은 비용으로, 더 일관되게 도달한 Agent가
더 좋은 Agent다

TroubleShooting

LLM 평가하면서 느낀점



🎲 결과가 매번 다를 수 있다



temperature=0이어도 미묘한 차이 존재
여러 번 실행 후 평균으로 비교하자

⚖️ LLM-as-Judge도 틀릴 수 있다



채점은 Rule 기반으로,
설명 품질 평가만 Judge로 분리하여 운영하자

🔍 평가 기준을 먼저 점검하라



잘못된 기준으로 Agent를 고치면
개선 방향 자체가 틀려진다

좋은 프롬프트는 좋은 기준에서 나온다

Wrap Up



Step 1

실패 사례를 모으면 패턴이 보이고, 패턴은 Rubric이 된다.

Step 2

Loop Engineering: Rubric이 있으면 LLM이 대신 채점하고, 반복해서 개선할 수 있다.

Step 3

Agent는 결과뿐 아니라 과정까지 봐야 어디를 고칠지 알 수 있다.

끝